# ТЕХНІКА СЕМАФОРІВ

Бінарні та загальні семафори

# Складність програмного вирішення проблеми взаємного виключення

- Алгоритм Деккера показав, що *програмне вирішення* проблеми взаємного виключення – *тільки за рахунок спільних змінних*, без використання спеціальних типів і структур даних – принципово можливе, проте *не є раціональним.*
- *Головною причиною складності* такого вирішення проблеми взаємного виключення є те, що перевірка значення спільної змінної і зміна цього значення – незалежні події і виконуються *порізну* окремими командами.
- В той час, коли один процес перевіряє умову X = ?, інший процес вже може змінювати значення X, і першому процесові про це *не відомо*. Крім того, такі рішення неефективні через *непродуктивну занятість* процесора при очікуванні події (busy waiting).

# Бінарні семафори

- Щоб змінити ситуацію, треба було вводити нові типи даних і операцій, в яких такі події були б *пов'язаними*, а виконання операцій – *непереревним*.
- *Бінарний семафор* — це не тільки ціла змінна s, значення якої можуть бути тільки 0 і 1, і не тільки дві операції: P(s) і V(s), що визначаються наступним чином:
  - P(s)≡if s≠0 then s-1 else await(s≠0)
  - V(s)≡ if s=0 then s+1
- Ці вирази не вичерпують повністю семантики семафорів і вимагають залучення структури даних *процесів*, що користуються спільними семафорами. Якщо позначити через *Q* множину процесів, що користуються семафором *s*, тоді маємо:
  - P(s)≡if s≠then s:=s-1 else Q:=Q ∪ {*this*};
  - V(s) ≡ if Q ≠ ∅ then Q:=Q\{any} else S:=1.

# Використання бінарних семафорів (1)

- Неперервність операцій P і V означає, що *тільки один процес* може виконувати *тільки одну операцію* над даним семафором в кожний момент.
- Семафор пов’язується з КС таким чином, що значення семафора 1 означає дозвіл входу в КС ("семафор відкрито"), а значення 0 ("семафор закрито") — очікування доти, доки це значення не стане 1. Початковим значенням семафора є значення 1, що дозволяє вхід в КС.
- Тепер проблема взаємного виконання вирішується дуже просто:
  - var s: sema; s:=1;
  - /*процес i */
  - P(s):
  - КС;
  - V(s)
- P і V операції утворюють "дужки",  що охоплюють критичну секцію КС.
- Семантика семафору гарантує, що процес, який виконав P-операцію, увійде в свою критичну секцію один, і при цьому жоден інший процес не зможе виконати таку ж операцію, що і означає *взаємне виключення*.

# Використання бінарних семафорів (2)

- Простота, з якою семафори вирішують проблему взаємного виключення, криється у тому, що вся *складність взаємодій переноситься на нижчий рівень операційної системи*, що саме і забезпечують семантику семафорів.
- Семафори або подібні до них засоби, увійшли до багатьох мов паралельного програмування і операційних систем. Вони стали ефективним засобом синхронізації *не тільки взаємовиключних операцій, але і кооперативних* дій процесів.
- Наприклад, відношення порядку між операціями $p_1$з процесу P1 і $p_2$ з процесу P2 таке, що $p_1 \rightarrow p_2$, тобто $p_2$ має виконуватись після $p_1$, може бути записане за допомогою семафорів у вигляді такого програмного фрагменту:
  - var S: sema ; S:=0;
  - parbegin(P1, P2)
  - P1:$p_1$; P2: P(s)
  - V(s) $p_2$
  - parend(P1, P2)

# Загальні семафори

- Загальним випадком семафорів є *лічильні (загальні)* семафори, значеннями яких можуть бути не тільки 0 і 1. P() і V() операції над загальними семафорами визначаються так:
  - $P(s) \equiv$ if $S>0$ then $S:=S-1$ else $Q:=Q \cup \{this\}$;
  - $V(s) \equiv$ if $Q \# \varnothing$ then $Q:=Q \setminus \{any\}$ else $S:= S+1$.
- На відміну від бінарного, загальний семафор може мати початковим значення будь-яке ціле $N \geq 0$.
- Е. Дійкстра показав, що синхронізаційні проблеми, що вирішуються за допомогою загальних семафорів, можуть бути вирішені, також і за допомогою бінарних семафорів.

# Модельна задача "постачальник–споживач" (проблема обмеженого буфера)

- В задачі маємо декілька процесів, що звуться *постачальниками*, які взаємодіють з групою інших процесів, що звуться *споживачами*, шляхом передачі їм "одиниць продукції" через *буфер обмеженої місткості.*
- При цьому і постачальники, і споживачі працюють паралельно і з невідомими швидкостями. Умовами правильної роботи (синхронізації) процесів є:
  1. неможливість доповнити повний буфер;
  2. неможливість вилучити з пустого буфера.
- В програмі, що подана нижче, ці дві умови реалізуються загальними семафорами, відповідно F і E. Для циклічного руху покажчика елементів в буфері використовується операція додавання по модулю mod N, де N – місткість буфера.

- var BUF: array [0.. N-1];
- Pointer: 0.. N-1; (*покажчик елемента, що вилучається*)
- Count: 0.. N-1; (*кількість елементів у буфері*)
- F, E: SEMA; (*загальні семафори*)
- ME: sema; (*бінарний семафор взаємного виключення*)
- COUNT:=0;  POINTER:=0; E:=N;  F:=0; ME:=1;
- (*постачальник*)
- var x  : (item)
- x:=…(*виробництво елемента*)
- P(E);
- P(ME);
- BUF[pointer
- +count) mod N]:=X;
- count:=count+1;
- V(ME);
- V(F)
- 

(*споживач*)

var x: (item)

P(F);

P(ME);

x:=BUF[pointer];

count:=count-1;

pointer:=(pointer+1) mod N

V(ME);

V(E);

…:=x; (*споживання елемента*)

# Використання загальних семафорів

- Семафор E (Empty) реалізує собою умову 2) (*неможливість вилучити з пустого буфера*); його поточне значення означає кількість вільних місць у буфері.
- Семафор F (Full) реалізує умову 1)(*неможливість доповнити повний буфер*); його поточне значення є кількість елементів розміщених у буфері.
- Бінарний семафор ME (Mutual Exclusion) потрібен для *взаємовиключного виконання* операцій над лічильником Count і покажчиком Point.
- Методологія використання загальних семафорів полягає у тому, що за допомогою V-операції процес *сигналізує* про здійснення умови (події) синхронізації, тоді як P-операція над даним семафором *здійснює захист* від неправильного чи передчасного використання цієї умови (події).
- Таким чином, загальний семафор розширює можливості бінарного семафора, який своїми двома станами (значеннями) може сигналізувати процесам тільки про умову "зайнятий/незайнятий" відносно якогось спільного ресурсу.